

WPF 开源项目收集

1. 项目特点

筛选 GitHub 项目：

- **App 级项目**（有 **App.xaml / MainWindow / Shell**），而不是单纯控件库
- 明确使用 **MVVM**（**ViewModels**、**INotifyPropertyChanged**、**Command**）
- UI 复杂度中等以上：导航、多页面、资源字典、主题、对话框、表格/树等
- 最近仍有人维护（commit / issue 活跃度），项目不能太老

2. 项目结构

2.1 单项目（小中型常见）

WPF “[新手教程](#)” 里也会强调 App.xaml 定义应用与资源，StartupUri 指向主窗口。

代码块

```
1  MyApp/  
2      App.xaml  
3      App.xaml.cs  
4      MainWindow.xaml  
5      MainWindow.xaml.cs  
6      Views/ .xaml .xaml.cs  
7      ViewModels/ .cs  
8      Models/ .cs  
9      Services/  
10     Converters/  
11     Behaviors/  
12     Resources/  
13         Styles.xaml  
14         Themes/
```

- **App.xaml**：定义应用级资源与启动配置（全局样式/主题/StartupUri）。
- **App.xaml.cs**：应用启动入口与生命周期代码（初始化 DI、全局异常、创建主窗口）。
- **MainWindow.xaml**：主窗口的界面结构（壳/布局/内容区）。
- **MainWindow.xaml.cs**：主窗口的代码隐藏（InitializeComponent + 少量 UI 行为/事件处理）。

2.2 多项目/模块化（Prism/大型项目常见）

Prism 的 samples 就是这种“由多个示例展示模块化/导航/MVVM”的典型代表。

代码块

```
1  MyApp.sln
2  src/
3      MyApp.Shell/           # 启动项目：App.xaml + ShellWindow
4      MyApp.Infrastructure/  # 通用服务、DI、日志、配置
5      MyApp.Modules.X/      # 功能模块：Views/ViewModels/Services
6      MyApp.Core/           # Domain Model、Contracts
```

2.3 文件类型

在 WPF 里很多文件会成对出现：Something.xaml 和 Something.xaml.cs。这是 WPF 的“标记 + 代码隐藏（code-behind）”组合，表示**同一个类被拆成两部分**。

- Something.xaml（UI 声明）→ Something.tsx（JSX 组件）
- Something.xaml.cs（UI 事件/行为）→ Something.tsx 内的事件处理，或抽到 hooks/*（如果逻辑较多）

不同子文件下的文件类型：

- **.xaml + .xaml.cs**：“视图 + 代码隐藏”，不仅迁 UI 结构，还要处理 code-behind 行为（最容易踩坑）
- **只有 .xaml**：要么“纯声明视图”，要么“资源字典”，即“最好迁”的纯 UI（或主题资源，需要转为 CSS/tokens）
- **只有 .cs**：“MVVM 的逻辑层”或“UI 辅助逻辑”，需要重新落到 React 的 store/hooks/services 或挪到后端/宿主

对于 MVVM 结构，是不是理论上 WPF 项目只需分这三层，再加上四个基础文件即可？

WPF 一般不叫“前后端分离”（那是 Web 领域的术语），但它有一个非常接近的工程思想：**UI（View/XAML）和业务逻辑（ViewModel/Model/Services）分离**，也就是 MVVM：

View 负责展示和绑定；ViewModel 负责状态和行为；Model 负责数据结构/领域对象。

- **App.xaml / App.xaml.cs**：应用入口、全局资源、启动窗口
- **MainWindow.xaml / MainWindow.xaml.cs**：主视图（.xaml.cs 通常只负责 InitializeComponent()，不写业务）
- **Views/**：.xaml 界面和 .xaml.cs，理论上每个页面（.xaml）的所有逻辑都可以写在对应的 .xaml.cs 中

- **ViewModels/**: 分担 View 层的逻辑；状态 + 命令 + 通知（INotifyPropertyChanged），如 MainViewModel
- **Models/**: DTO，负责前后端通信；纯数据对象（POCO），如 Person
- **Controls/Converters/Services/Properties**: 进一步拆分 ViewModel 层（及其他层）的逻辑，以及后端逻辑

3. 项目收集（WPF MVVM）

Awesome WPF: <https://github.com/Carlos487/awesome-wpf>

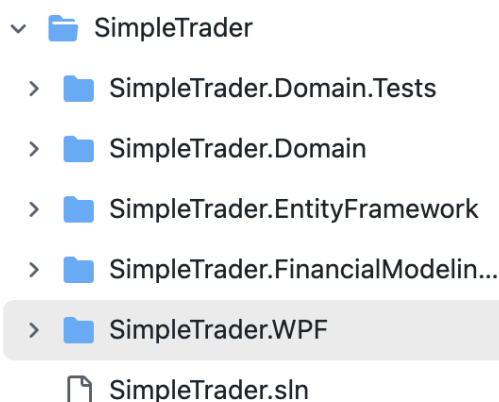
- [SimpleTrader](#) - A full stack WPF MVVM trading application.
- [The World's Simplest C# WPF MVVM Example](#) - A simple MVVM example using WPF and C# 9.

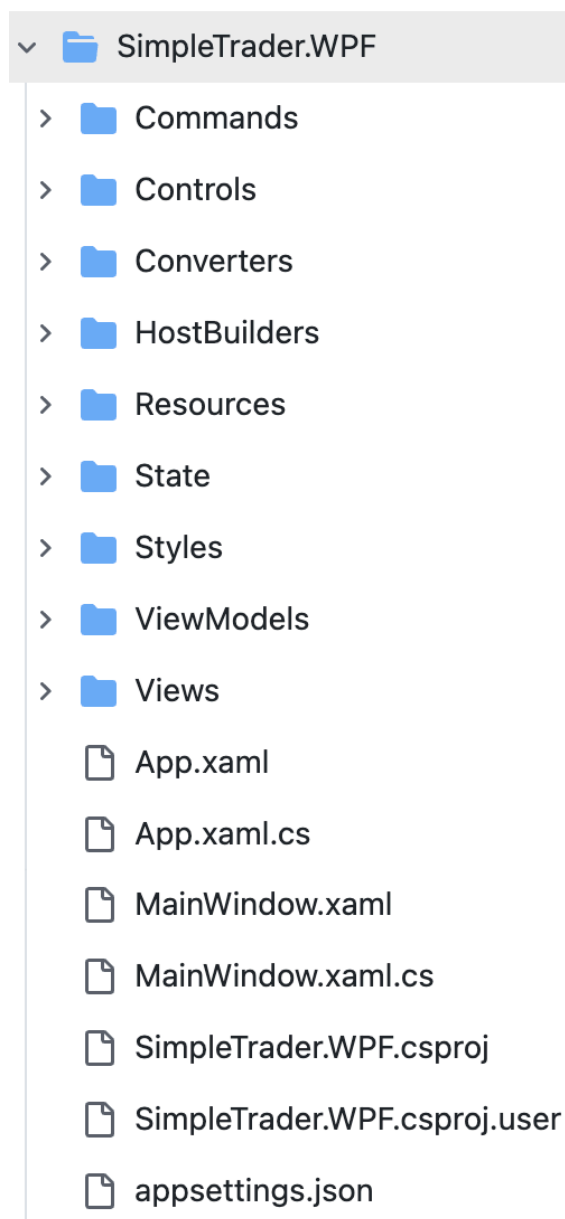
检索 WPF MVVM，要么太老要么 star 太少

检索 WPF，搜索 App.xaml 文件，再看架构是否符合 MVVM（View, ModelView）

3.1 SingletonSean/SimpleTrader（2021-2023）

A full stack WPF MVVM trading application.





需要将 SimpleTrader.WPF/ 迁移至 React 项目对应的 web-ui/:

1. 若按照文件粒度，由于和 MVVM 的文件组织方式不同，迁移后代码不一定会一一对应。所以是不是应该先设计一套逻辑分析源代码库，得到：

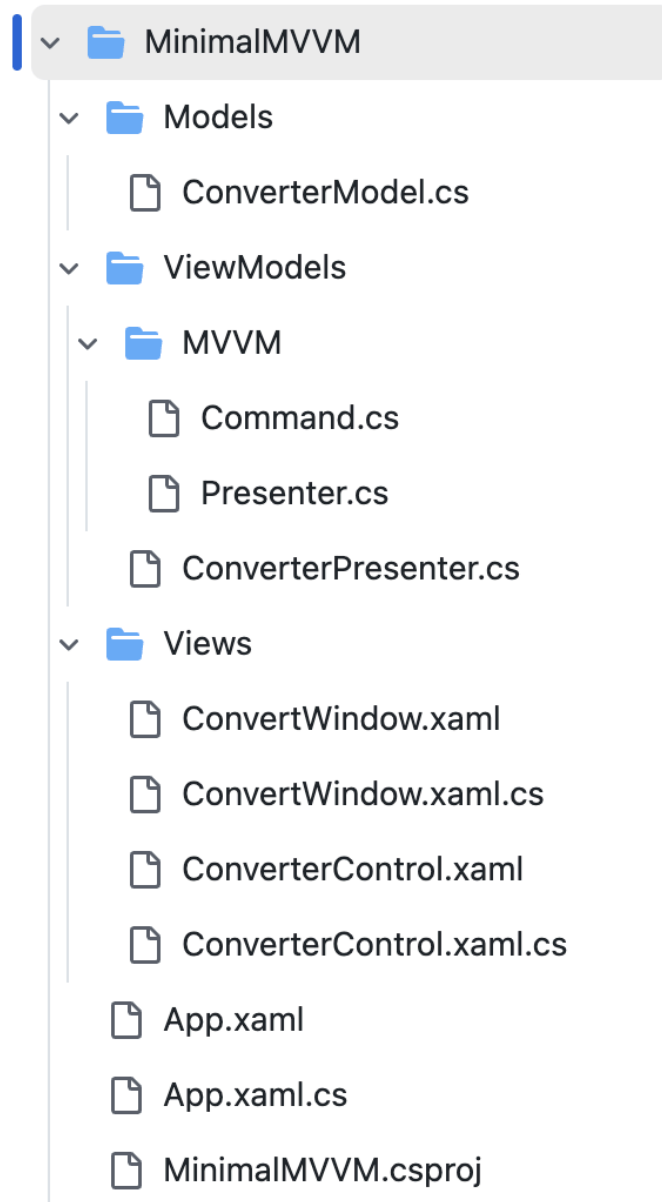
- 目标库文件结构：包含哪些文件，以及组织好所有文件的层级关系
- 迁移文件映射表：目标库的每一个文件是从源库的哪些文件生成

如果可行，后续逐步生成各文件即可。但是感觉这么做困难点很多，需要输入整个项目上下文会超，是否可以通过 Agent 自动总结各文件或主动加 description 解决？

2. 若按照构建依赖图再自底向上迁移的思路，依赖图的粒度是文件级吗？后端依赖图一般使用类似 tree-sitter 等静态分析工具，前端依赖图该如何构建？

3.2 [MarkWithall/worlds-simplest-csharp-wpf-mvvm-example](#) (2021)

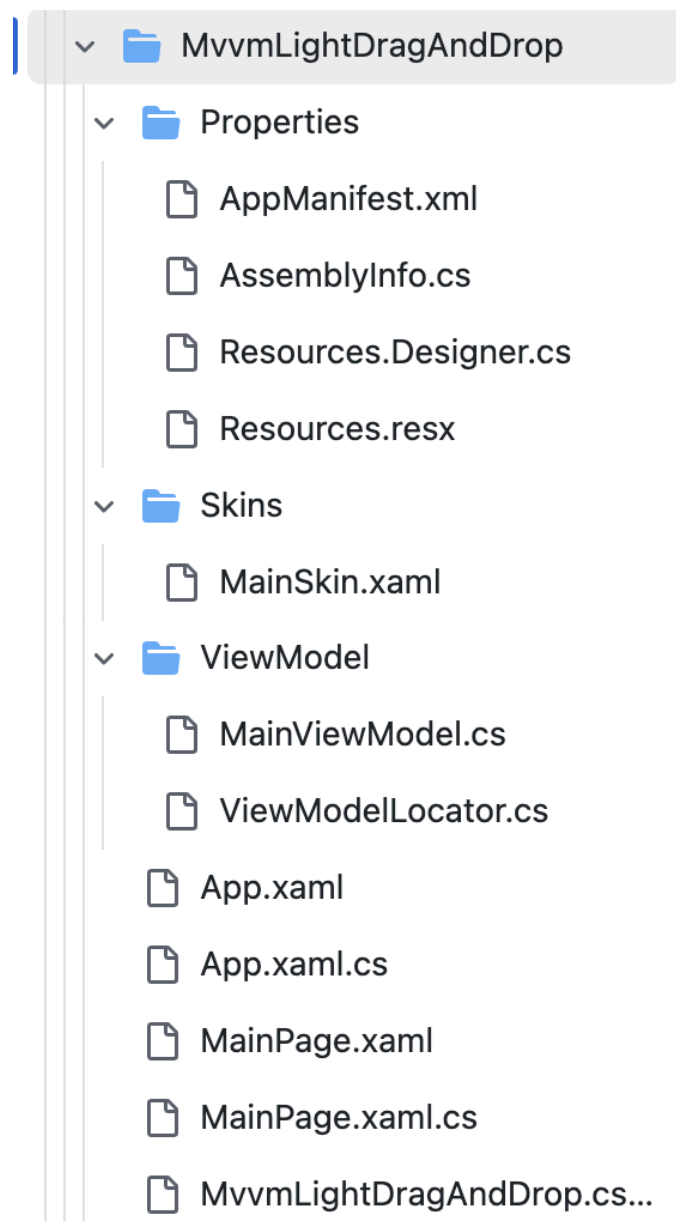
The World's Simplest C# WPF MVVM Example as described [here](#). A C# 9 (.NET 5.0) version of the code can be found in the [C#9.0 branch](#).



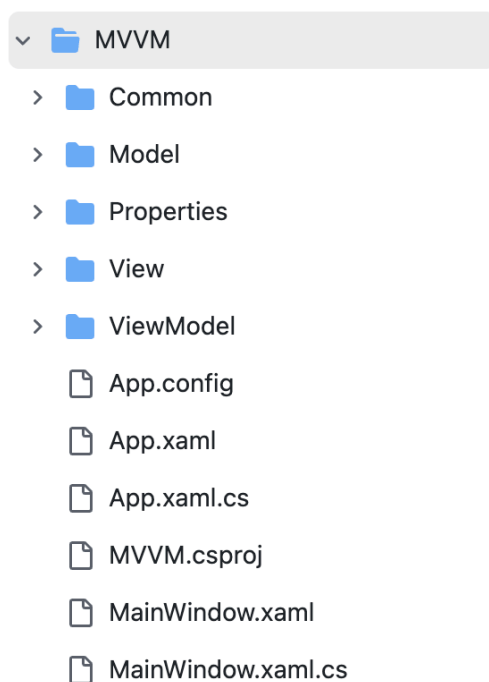
3.3 lbugnion/mvvmlight (2021, 停止维护)

该工具包的主要目的是加速在 Xamarin.Android、Xamarin.iOS、Xamarin.Forms、Windows 10 UWP、Windows Presentation Foundation (WPF)、Silverlight、Windows Phone 上创建和开发 MVVM 应用程序。

Samples/MvvmLightDragAndDrop/MvvmLightDragAndDrop



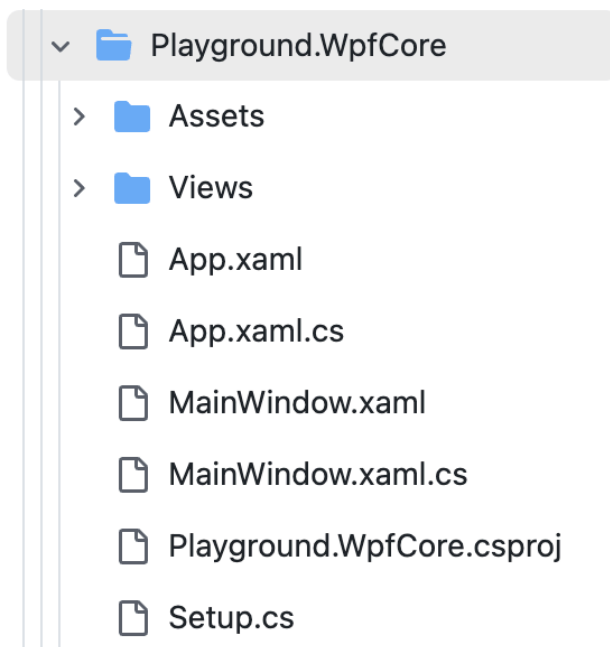
3.4 944095635/MVVM (2021, 不规范)



3.5 🌟MvvmCross/MvvmCross (框架, 2025, 3.9k)

MvvmCross 是一个主观的跨平台 MVVM 框架。它使开发人员能够在 .NET 生态系统中使用 MVVM 模式创建应用程序。我们支持 Android、iOS、MacCatalyst、TvOS、macOS、WinUI、WPF。使用 MvvmCross 可以通过允许您在平台之间共享行为和业务逻辑来实现更好的代码共享。

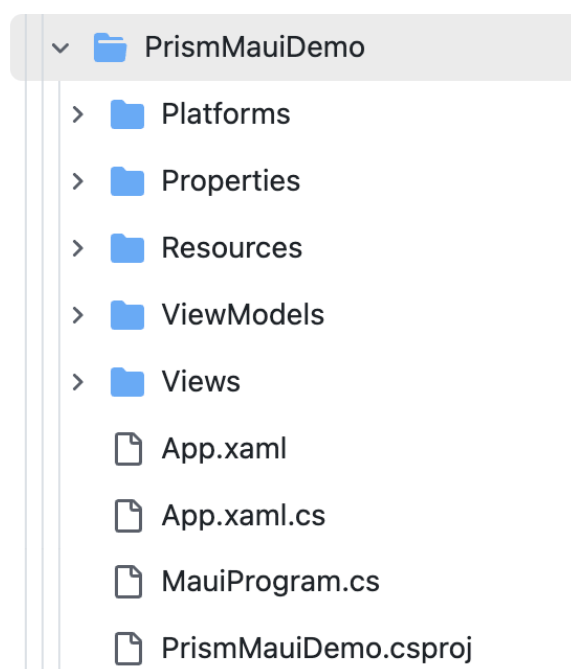
[Projects/Playground/Playground.WpfCore](#)



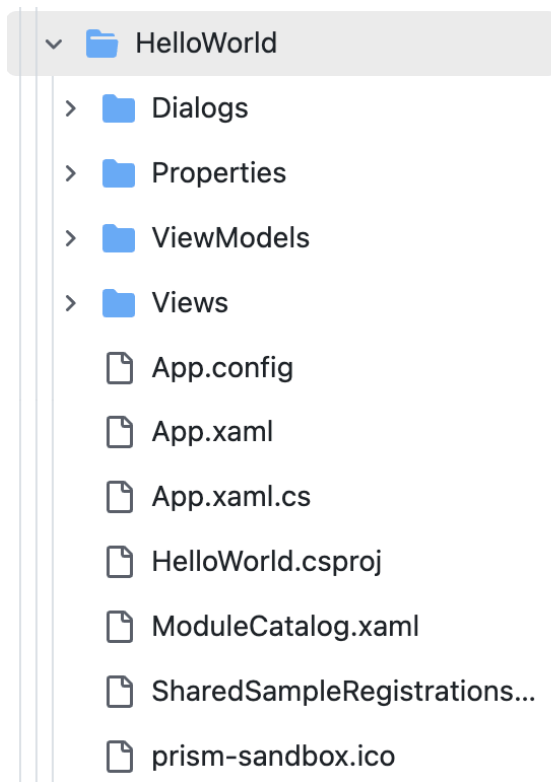
3.6 🌟PrismLibrary/Prism (框架, 2025, 6.7k)

Prism 是微软官方推荐的 MVVM 模块化框架, 适用于 WPF、Uno/WinUI 等 XAML 应用。它提供了区域导航、事件聚合器和依赖注入等机制, 可帮助构建松耦合、可维护的 WPF UI。

[e2e/Maui/PrismMauiDemo](#)



[e2e/Wpf/HelloWorld](#)



3.7 CSharpDesignPro/Page-Navigation-using-MVVM (2022)

WPF - Page Navigation using MVVM

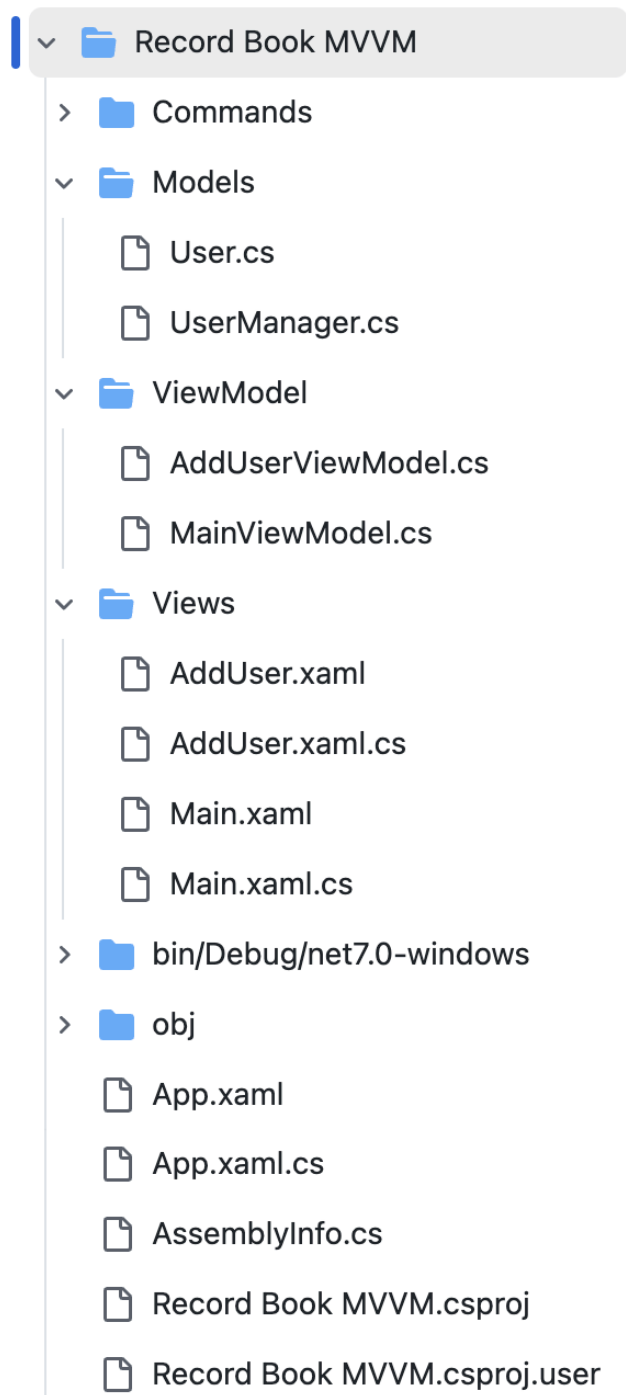
 CSharpDesignPro Update README.md		a4c42a2 · 3 years ago	 5 Commits
 Fonts	Add files via upload	3 years ago	
 Images	Add files via upload	3 years ago	
 Model	Add files via upload	3 years ago	
 Source Code	Add files via upload	3 years ago	
 Styles	Add files via upload	3 years ago	
 Utilities	Add files via upload	3 years ago	
 View	Add files via upload	3 years ago	
 ViewModel	Add files via upload	3 years ago	
 App.xaml	Add files via upload	3 years ago	
 App.xaml.cs	Add files via upload	3 years ago	
 LICENSE	Initial commit	3 years ago	
 MainWindow.xaml	Add files via upload	3 years ago	
 MainWindow.xaml.cs	Add files via upload	3 years ago	
 README.md	Update README.md	3 years ago	

3.8 RJCodeAdvance/Login-In-WPF-MVVM-C-Sharp-and-SQL-Server (2022)

How to create a Login Form Implementing the MVVM pattern, WPF, C-Sharp and SQL Server

Name	Last commit message	Last commit date
..		
CustomControls	Add files via upload	3 years ago
Images	Add files via upload	3 years ago
Models	Add files via upload	3 years ago
Properties	Add files via upload	3 years ago
Repositories	Add files via upload	3 years ago
ViewModels	Add files via upload	3 years ago
Views	Add files via upload	3 years ago
bin/Debug	Add files via upload	3 years ago
obj/Debug	Add files via upload	3 years ago
App.config	Add files via upload	3 years ago
App.xaml	Add files via upload	3 years ago
App.xaml.cs	Add files via upload	3 years ago
WPF-LoginForm.csproj	Add files via upload	3 years ago

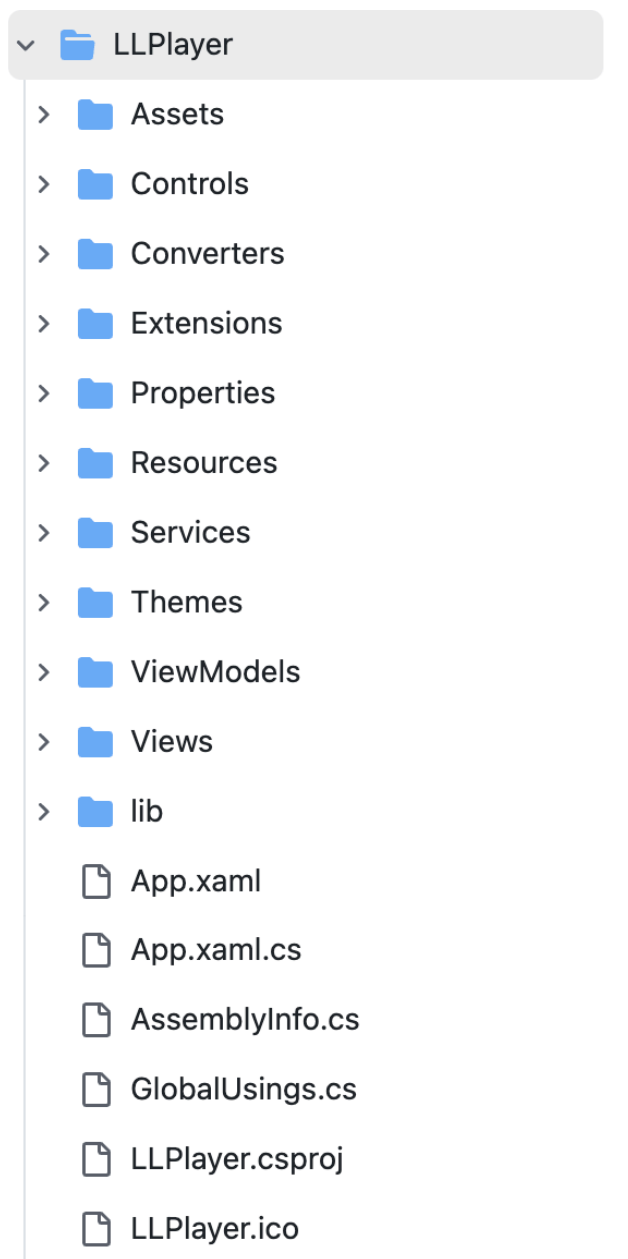
3.9 TacticDevGit/Record-Book-App-WPF-MVVM (2023)



3.10 🌟umlx5h/LLPlayer (2024, 2.2k)

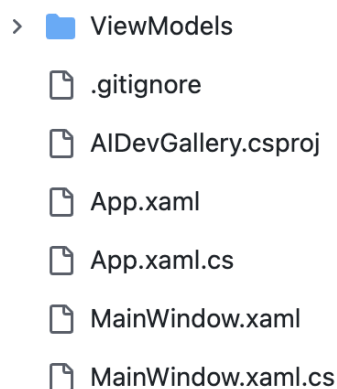
一款专注于字幕相关功能的视频播放器，包括双语字幕、AI 生成字幕、实时翻译、单词查询等功能！

[LLPlayer](#)



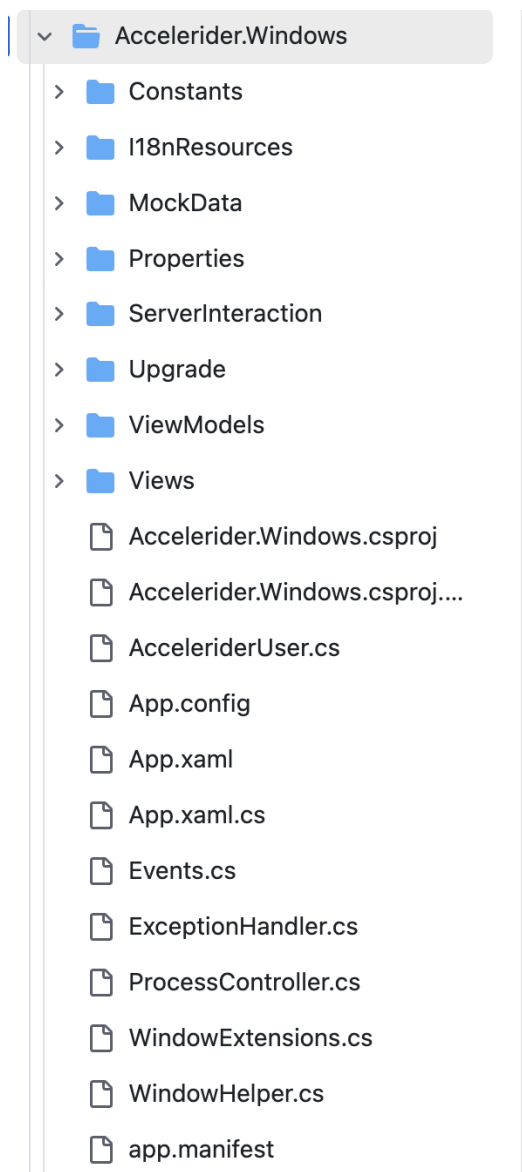
3.11 [microsoft/ai-dev-gallery](#) (2025)

专为 Windows 开发者设计，AI 开发画廊帮助将 AI 功能集成到应用程序和项目中。



3.12 [Accelerider/Accelerider.Windows](#) (2024, 1.5k)

Accelerider.Windows 是基于 Prism 的 WPF 外壳应用，采用模块化和 MVVM 架构构建加速器客户端。



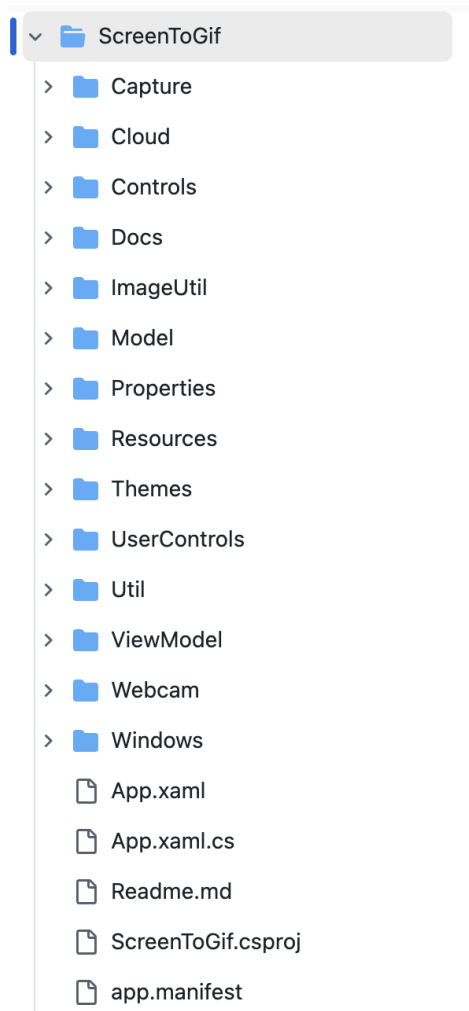
4. 真实应用（GPT）

4.1 NickeManarin/ScreenToGif (2025, 26.1k)

功能完备的屏幕录像与 GIF / 视频编辑器，使用 WPF 和 MVVM 架构，支持录屏、帧编辑和导出

[ScreenToGif](#)

Windows 可能是 View 层，app.manifest 可能是 MainWindow.xaml.cs

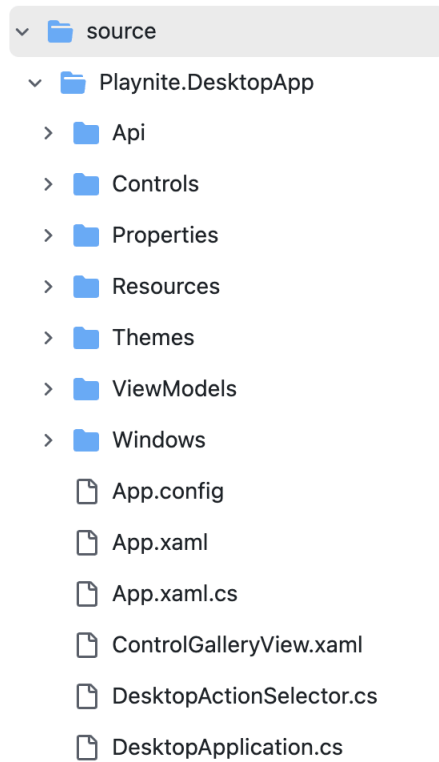


4.2 🌟JosefNemec/Playnite (2025, 12.1k)

开源游戏库管理器与启动器，采用 WPF 界面和 MVVM 模式，可集成 Steam、GOG 等多种平台；仓库主题包含 wpf

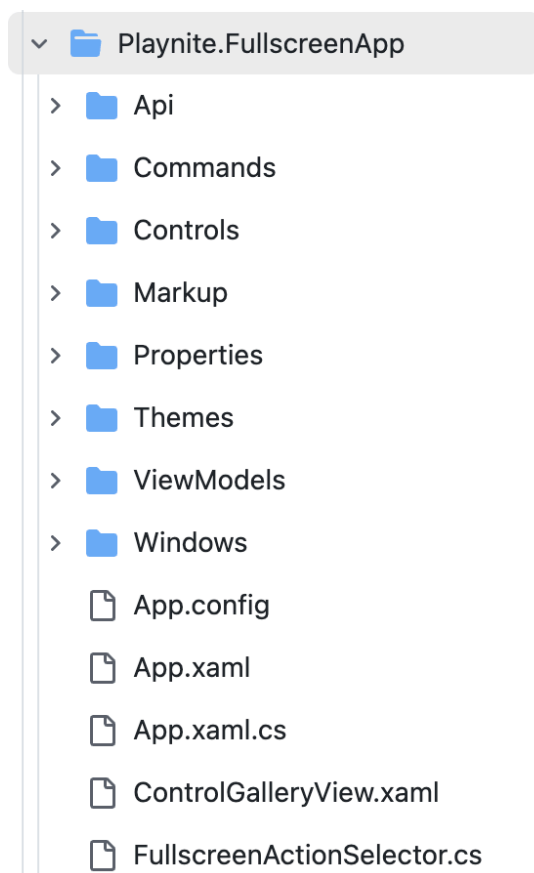
[source/Playnite.DesktopApp](https://github.com/JosefNemec/Playnite.DesktopApp)

Windows 可能是 View 层，且 MainWindow.xaml.cs 被放入 Windows/



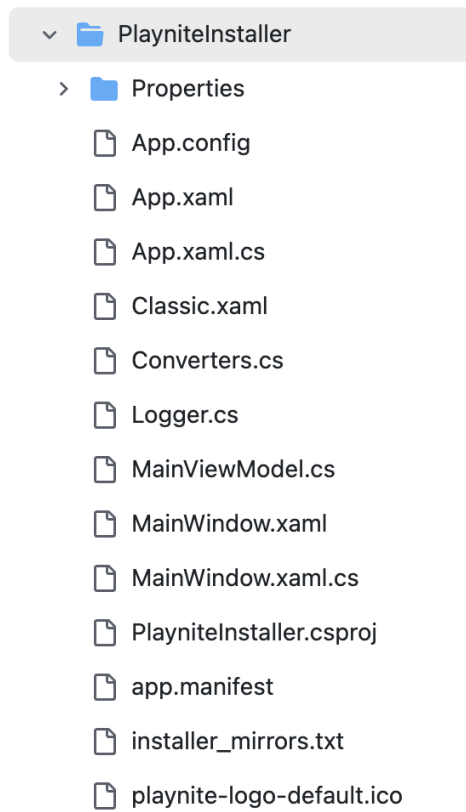
[source/Playnite.FullscreenApp](#)

Windows 可能是 View 层，且 MainWindow.xaml.cs 被放入 Windows/



[source/Tools/PlayniteInstaller](#)

较为简单，没有明显的 MVVM 结构，但是四个基础文件齐全

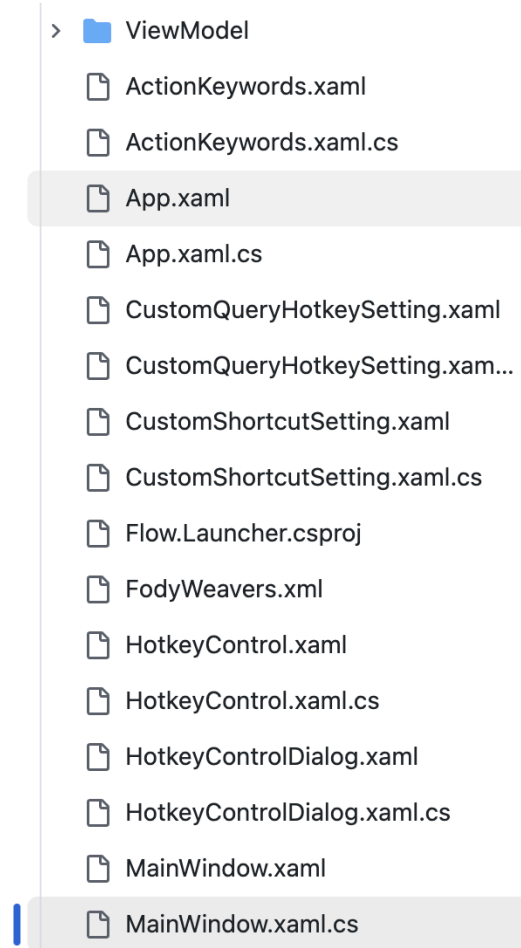


4.3 🌟Flow-Launcher/Flow.Launcher (2025, 12.5k)

快速启动器和文件搜索工具，使用 WPF/MVVM 构建，支持插件扩展

[Flow.Launcher](#)

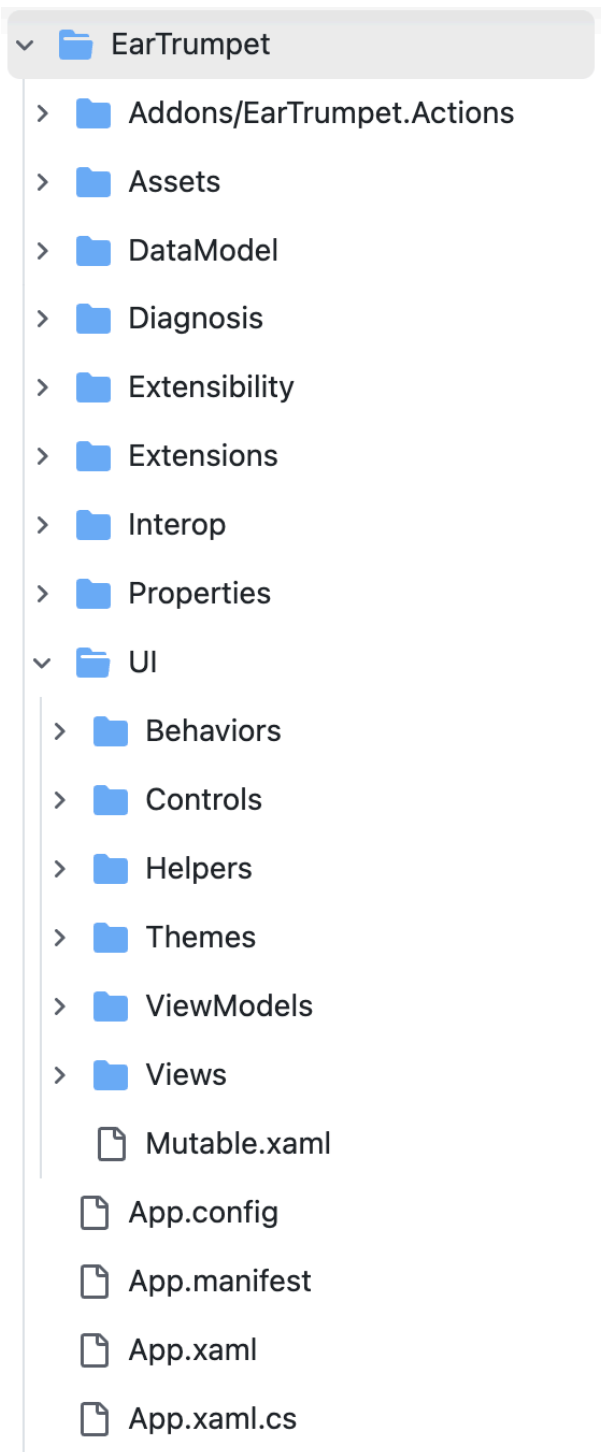
View 层被放在和 App.xaml/MainWindow.xaml 同级；Flow.Launcher/Converters 可能是 Model 层



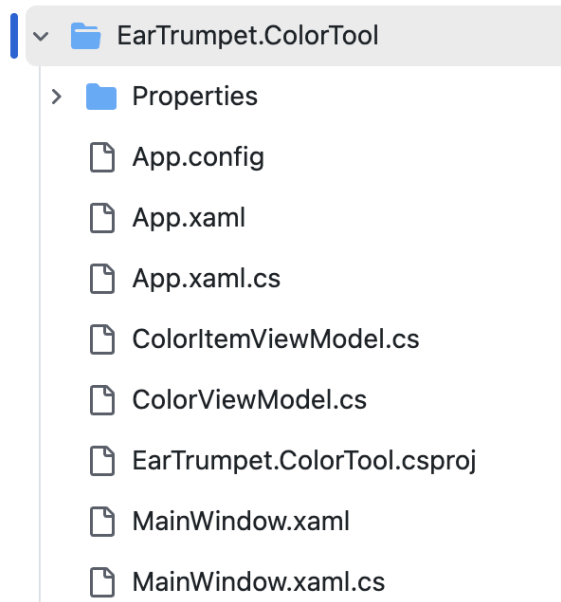
4.4 File-New-Project/EarTrumpet (2025, 10.3k)

Windows 音量控制增强应用，采用 WPF 构建，可对每个应用进行混音控制；主题标签包含 audio、wpf 等

[EarTrumpet](#)



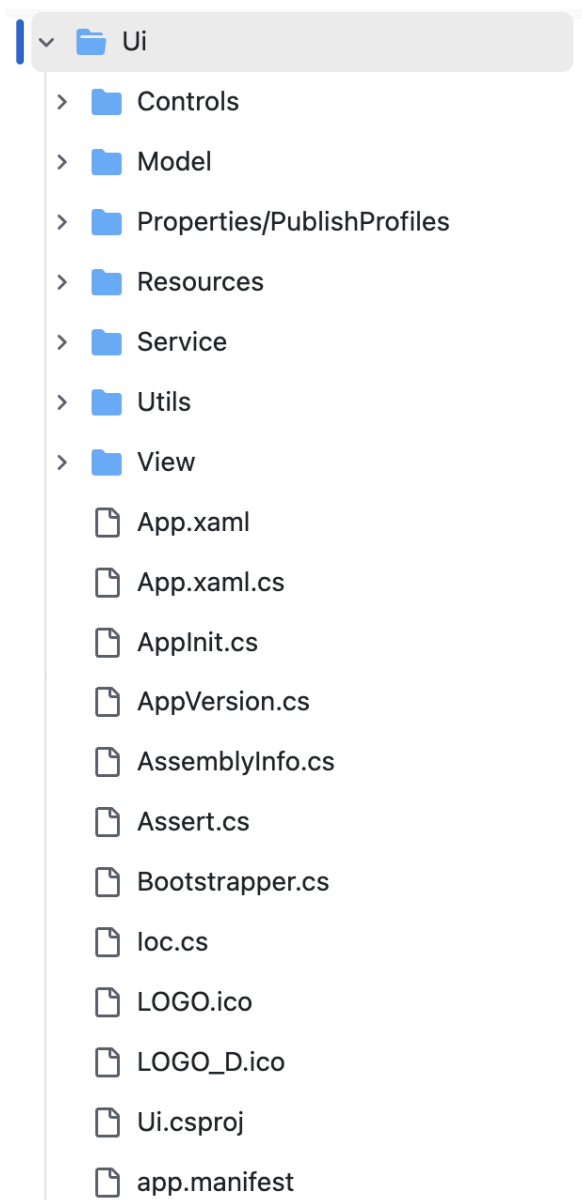
EarTrumpet.ColorTool



4.5 1Remote/1Remote (2025, 2.7k)

远程访问/终端管理器，支持 RDP/VNC/SSH 等，仓库描述中明确是 WPF 应用。

Ui



4.6 🌟VisualHFT/VisualHFT (2025, 0.9k)

金融市场微结构数据可视化工具，基于 WPF/C#，其 topics 包括 mvvm 和 wpf-application，说明采用 MVVM 架构

View	updated oxyplot-skiasharp package for much better perf...	4 months ago
ViewModel	Improved string handling (formatting) in studies	last week
VisualHFT.Commons.WPF	fixed bugs	3 months ago
VisualHFT.Commons	CustomObjectPool refactoring, improving reliability and p...	last week
VisualHFT.DataRetriever.TestingFramework	Refactor OrderBookSnapshot for performance and clarity,...	3 weeks ago
VisualHFT.Plugins	Kraken nuget packages upgrade	last week
VisualHFT.TriggerService.TestingFramework	packages update	last month
WS_input_json	getting ready for github	3 years ago
demoTradingCore	packages upgrade	last week
docImages	Add files via upload	2 years ago
docs	architecture diagram	5 months ago
.gitattributes	Update .gitattributes	2 years ago
.gitignore	- Migration to Latest .NET 7.0 Framework	2 years ago
App.config	Core system performance updates and bug fixes.	last year
App.xaml	Core system performance updates and bug fixes.	last year
App.xaml.cs	Refactor OrderBookSnapshot for performance and clarity,...	3 weeks ago
CONTRIBUTING.md	Update CONTRIBUTING.md	6 months ago
FodyWeavers.xml	Major changes: see release notes	2 years ago
FodyWeavers.xsd	Major changes: see release notes	2 years ago
GlobalStyle.xaml	updated oxyplot-skiasharp package for much better perf...	4 months ago
LICENSE.txt	Add README.md and LICENSE.txt.	3 years ago
MainWindow.xaml	Core system performance updates and bug fixes.	last year
MainWindow.xaml.cs	Core system performance updates and bug fixes.	last year

Contributing

Activity

Custom properties

975 stars

38 watching

195 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors 5

Deployments 64

github-pages last week

+ 63 deployments

Languages

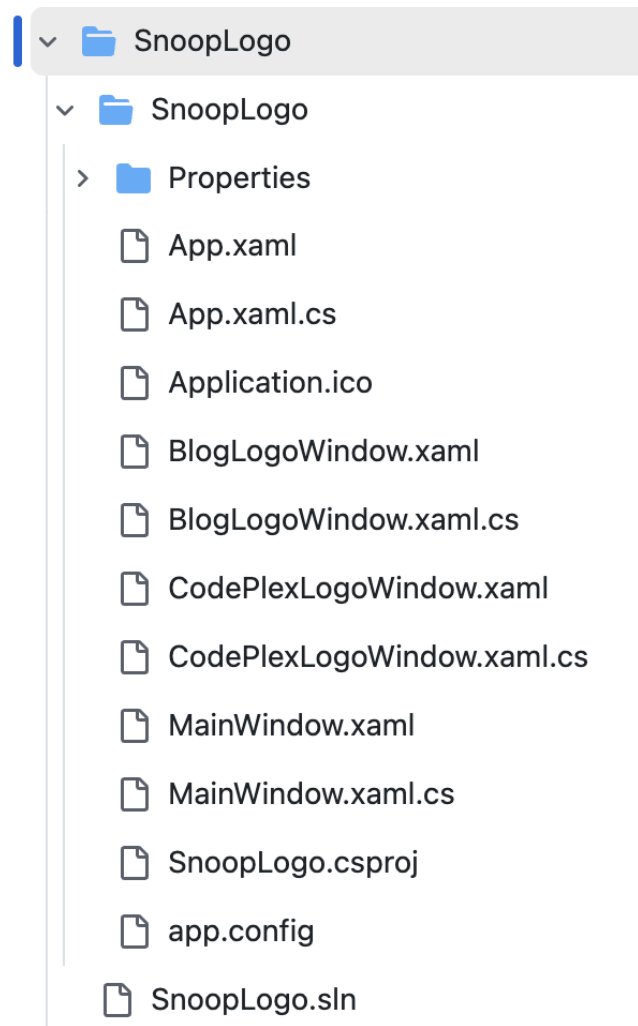
C# 100.0%

4.7 snoopwpf/snoopwpf (2025, 2.4k)

WPF 应用调试和可视化工具，可窥探运行中应用的视觉树、资源等，采用 WPF 构建

SnoopLogo

较为简单，没有明显的 MVVM 结构，但是四个基础文件齐全



4.8 🌟SlimeNull/OpenGptChat (2025, 0.1k)

开源 ChatGPT 桌面客户端，基于 WPF + MVVM 架构，支持自定义 API Key 等

[OpenGptChat](#)

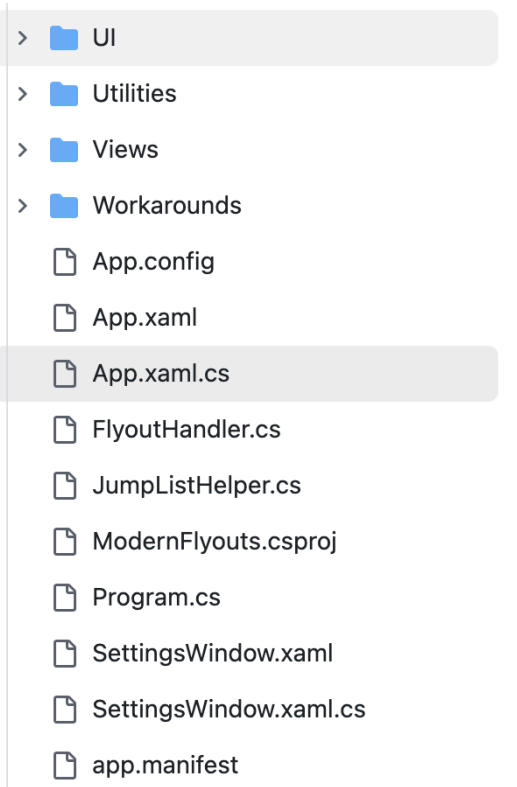
- >  ColorModes
- >  Controls
- >  Converters
- >  Languages
- >  Markdown
- >  Models
- >  Properties
- >  Resources
- >  Services
- >  Themes
- >  Utilities
- >  ViewModels
- >  Views
-  App.xaml
-  App.xaml.cs
-  AppWindow.xaml
-  AppWindow.xaml.cs
-  AssemblyInfo.cs
-  EntryPoint.cs
-  OpenGptChat.csproj
-  app.manifest

4.9 ModernFlyouts-Community/ModernFlyouts (2025, 4k)

一款 Fluent Design 风格的音量/亮度飞出面板替代品，基于 WPF/MVVM。不过该仓库已标记为“archived”。

ModernFlyouts

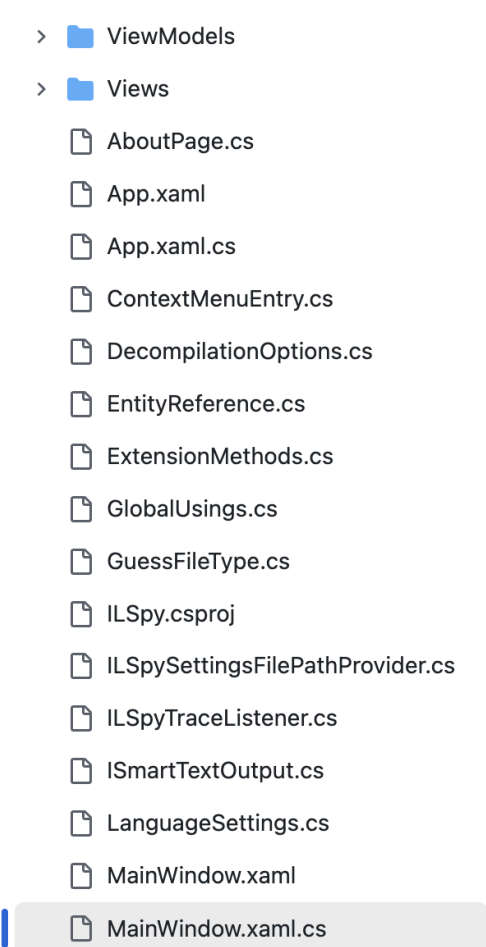
UI 可能是 ViewModel 层，SettingsWindow.xaml.cs 可能是 MainWindow.xaml.cs



4.10 🌟 [icsharpcode/ILSpy](#) (2025, 24.3k)

开源 .NET 反编译器，其 GUI 前端基于 WPF；虽然同时支持多平台，但作为 WPF 应用示例值得关注

[ILSpy](#)



5. 其他问题

1. 只需要管 UI 迁移，不需要管后端逻辑？相关逻辑直接让大模型 mock 或迁移后手动 mock？
2. WPF 相关文件夹藏太深了，手动逐个提取吗？不需要后端逻辑，其他无关文件夹可以不用管？
3. 把所有收集到的项目结构归一化为 2.1 类似的结构是否可行？如果该流程手动完成，写论文里是不是太简单了？
4. 如何评估迁移效果：编译通过率（组件级、页面级）、界面相似度（截图大模型打分）
 - 使用 ESLint 检查语法正确性（使用 Vite React-TS 构建自带 eslint.config.js），运行 `npm run lint` 会对项目里所有 .ts/.tsx 文件启用“JS 推荐规则 + TS 推荐规则 + React Hooks 规则 + React Refresh 规则”，并忽略 dist/。
 - 定义 UI 界面组件的状态与状态转移，验证迁移前后是否一致

